

Humanoid Stand-Up with Deep Reinforcement Learning

Juan Liang Liao
Department of Artificial Intelligence
San Jose State University
San Jose, CA, USA

Kirubiel Gebreselassie
Department of Artificial Intelligence
San Jose State University
San Jose, CA, USA

Sravan Peethani
Department of Artificial Intelligence
San Jose State University
San Jose, CA, USA

Tyler Jones
Department of Artificial Intelligence
San Jose State University
San Jose, CA, USA

Abstract—The aim of this project is to compare the performance of multiple reinforcement learning algorithms on the humanoid stand up problem, where a humanoid robotic agent is trained to stand up from a prone position. For the environment, we use Gymnasium’s MuJoCo HumanoidStandup-v5 for training and evaluation. We focus our analysis on two algorithms, Proximal Policy Optimization (PPO) and Soft Actor Critic (SAC). We will begin with Proximal Policy Optimization (PPO) as the baseline algorithm, since it’s known to be stable and effective for continuous motion problems. We then implement Soft Actor Critic (SAC) as a comparative model, which adds a bit more exploration and can often learn faster by balancing reward and randomness. Because the humanoid standup task gives very few rewards, mostly when the robot fully stands up, we will add some intermediate rewards to guide learning. These smaller rewards are given when the agent makes partial progress, like lifting its torso or holding steady posture, helping it learn more smoothly instead of struggling with a long stretch of zero feedback. The performance of both algorithms is evaluated using mean episodic returns and several training diagnostics including critic loss, policy entropy/temperature and the PPO clip fraction. Additionally, we will report training learning curves to analyze sample efficiency over time. Through comparative analysis, the projects aims to highlight how algorithm choice and reward design affect learning dynamics, stability, and overall task performance in humanoid motors.

I. INTRODUCTION / LITERATURE REVIEW

Humanoid motion control is a long-standing challenge in robotics, involving high-dimensional state spaces, unstable dynamics, and sparse feedback. Reinforcement Learning (RL) has become an effective approach for learning complex motor behaviors by directly optimizing control policies through interaction.

Early studies such as Tassa et al. (2012) demonstrated how dynamic motor sequences can be synthesized using trajectory optimization, laying the groundwork for learning humanoid movements like standing up and balancing. Later, Schulman et al. (2017) introduced Proximal Policy Optimization (PPO), which significantly improved the stability and efficiency of policy-gradient methods. PPO became a standard baseline for continuous-control tasks because of its clipped objective function that prevents destructive policy updates.

Building upon these foundations, Haarnoja et al. (2018) proposed Soft Actor-Critic (SAC), an off-policy actor-critic algorithm that incorporates entropy regularization to encourage exploration. SAC has proven especially effective in MuJoCo environments where balancing exploration and exploitation is critical.

Further work on humanoid control, such as “An End-to-End Reinforcement Learning Agent for Fall Recovery and Stand-Up of Humanoid Robots” (IEEE, 2023), explored how agents can autonomously recover from fallen states and return to balance through deep RL. These studies emphasize the importance of reward shaping and stable training strategies for physically plausible behaviors. The Gymnasium platform (Farama Foundation, 2023) provides standardized RL environments, including HumanoidStandup-v4, built on MuJoCo physics. Using these environments enables reproducibility and fair benchmarking of RL algorithms in simulated robotics.

Collectively, this body of work motivates our project’s focus on comparing PPO and SAC for humanoid stand-up control while addressing reward sparsity through intermediate shaping. By leveraging prior advances in continuous-control RL and simulation frameworks, our study aims to evaluate how algorithm design and reward formulation influence the efficiency, stability, and realism of learned humanoid behaviors.

II. DATASETS AND EXPERIMENTAL

This project does not require a dataset and experimental setup. This is because google colab and gymnasium HumanoidStandup-v5 environment are being used.

Repository: <https://github.com/JDLiang100/CMPE-260-ProjectHumanoidStandup>

A. Data Source

This project does not use any external datasets. All training data is generated through interactions within the Gymnasium HumanoidStandup-v5 environment, which runs on the MuJoCo physics engine. Each training episode produces sequences of observations, actions, rewards, and termination

flags as the agent learns to stand up from a lying position. The environment inherently provides continuous, high-dimensional state and action spaces, making it suitable for reinforcement learning experiments in complex motor control.

B. Environment Configuration

For the environment, we used Google Colab, which has most of the packages required for the project built into it. To run the Humanoid Stand Up environment, we also added the MuJoCo physics engine with the command, `pip install gymnasium[mujoco]`.

III. METHODOLOGY

A. Proximal Policy Optimization (PPO)

This project implements a custom Proximal Policy Optimization (PPO) algorithm from scratch using PyTorch to train a humanoid agent for the stand-up task. PPO is chosen for its balance between stability and performance in continuous-control environments such as HumanoidStandup-v5.

The actor network is the policy model that decides which actions the humanoid should take based on its current full-body state. It has two fully connected layers with 256 hidden units each and uses ReLU activation functions to learn complex patterns in the state space. The output layer produces the mean of the action distribution and a trainable log standard deviation, forming a Gaussian policy. This setup enables the agent to generate continuous actions and explore effectively by sampling from the action distribution instead of always choosing the same action.

The critic network predicts how good a particular state is under the current policy, acting as a baseline to guide learning. It takes the observation vector as input and passes it through two fully connected layers with 256 hidden units each, using ReLU activations. The network then outputs a single scalar value that estimates the expected return from that state. This value helps the actor network make better decisions by providing feedback on which states are more favorable for achieving higher rewards.

The PPO agent trains both the actor and critic networks using two main loss functions. The **actor loss** uses a clipped objective that limits how much the policy can change at each update. This helps keep training stable and prevents sudden drops in performance. The **critic loss** is a mean-squared error (MSE) between the predicted state values and the actual rewards received, helping the critic learn to estimate returns more accurately. By combining these two losses, the PPO agent improves its policy while keeping the learning process smooth and stable.

Actor Loss:

$$r_i(\theta) = \frac{\pi_{\theta'}(a_i | s_i)}{\pi_{\theta}(a_i | s_i)}$$

Fig. 1. Importance Sampling Ratio

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

Fig. 2. Clipped surrogate objective for PPO

Critic Loss:

$$\mathcal{L}_{CRITIC}(\phi) = \frac{1}{N} \sum_{i=1}^N (V_{\phi}(s_i) - \hat{V}(s_i))^2$$

To stabilize learning and reduce variance, the algorithm uses Generalized Advantage Estimation (GAE). This method computes smoothed estimates of the advantage function, balancing bias and variance through the parameter γ ($\gamma = 0.95$).

Both the actor and critic parameters are updated using the Adam optimizer with a learning rate of 3×10^{-4} . Training proceeds in batches of on-policy samples collected from the environment, with multiple epochs of gradient updates performed after each rollout.

B. 3.2 Soft Actor Critic (SAC)

Soft Actor-Critic (SAC) is an off-policy reinforcement learning algorithm designed to balance high return performance with robust exploration. Unlike deterministic approaches, SAC explicitly learns a stochastic policy that maximizes both the expected reward and the entropy of the action distribution. This formulation encourages the agent to prefer uncertain and exploratory behaviors, rather than converging prematurely to the safest local optimum. In continuous control environments, particularly those with high-dimensional action spaces such as humanoid locomotion tasks, this entropy-augmented objective allows SAC to explore a broader region of the policy space and avoid collapsing into risk-averse behaviors.

The actor network outputs a squashed Gaussian policy. It takes the state as input, passes it through two ReLU layers, and produces the mean and log standard deviation of the action distribution. The log-std is clamped between 20 and 2 to avoid unstable variance, and its bias is initialized to 0.5 to encourage early exploration. Actions are sampled using the reparameterization trick and squashed with *tanh*, ensuring they stay within the environment bounds. A correction term is applied to the log-probability to account for this squashing, so the entropy term remains mathematically valid.

The critic is implemented as a double Q-network. Each critic receives the concatenation of state and action and predicts a scalar Q-value through two hidden layers. During training, SAC uses the minimum of the two Q-values to reduce overestimation bias and provide a more conservative learning signal. A target critic is maintained and updated via soft parameter interpolation, which stabilizes temporal difference learning and prevents value drift.

SAC jointly optimizes three objectives: the critic loss, the actor loss, and the entropy temperature. The critic minimizes the mean-squared error between its Q-values and a bootstrapped target that includes both reward and a policy entropy term. The actor optimizes a loss of $\log(\pi(a_i|s_i)) - Q(s_i, a_i)$, encouraging actions that have high Q-values while maintaining stochasticity. The temperature parameter is updated automatically to match a target entropy, preventing the policy from becoming prematurely deterministic.

Transitions collected during interaction are stored in a replay buffer and sampled in mini-batches to decorrelate updates. Both actor and critic parameters are optimized using Adam, with optional gradient clipping to improve stability. The target critic network is updated after each step using a soft update parameter α , allowing it to track the learned critic slowly rather than copying weights abruptly. This combination of replay, double critics, entropy tuning, and soft targets enables stable learning in high-dimensional continuous control tasks.

$$L_{ACTOR}(\theta) = \frac{1}{N} \sum_{i=1}^N \alpha \log \pi(a_i|s_i) - \min(Q_{\phi_1}(s_i, a_i), Q_{\phi_2}(s_i, a_i))$$

Fig. 3. Actor Loss

$$L_{CRITIC}(\phi_j) = \frac{1}{N} \sum_{i=1}^N (Q_{\phi_j}(s_i, a_i) - y_i)^2 \quad \text{for } j = 1, 2$$

Fig. 4. Critic Loss

$$L_{\alpha} = \frac{1}{N} \sum_{i=1}^N \alpha (-\log \pi(a_i|s_i) - \mathcal{H}_{\text{target}})$$

Fig. 5. Entropy Temperature

C. 3.3 Algorithm Comparison

Both PPO and SAC aim to solve the same continuous-control humanoid stand-up task, but they approach learning very differently. PPO optimizes the policy directly using on-policy samples and a clipped objective, which produces stable but slower updates and often requires multiple environment rollouts to improve performance. In contrast, SAC is off-policy and learns from replayed experience, making it more sample-efficient and better at handling sparse or unstable rewards. PPO

tends to behave conservatively and can settle into safe local optima, while SAC explicitly promotes exploration through entropy maximization, encouraging the agent to try novel, high-risk actions such as leaning, shifting body mass, or pushing off the floor. For the humanoid task, this difference is significant: PPO provides steady improvement, but SAC generally discovers more effective standing strategies because it tolerates instability and rewards exploratory motion.

IV. RESULTS AND ANALYSIS

The agent was trained for 5000 episodes using Soft Actor-Critic(SAC) and Proximal Policy Optimization(PPO). For the SAC and PPO implementation, we ran several models with different structures and hyperparameters. For the results, the average reward fluctuated within 50,000 to 80,000 and our best evaluation reward peaked around 120,000. Upon further evaluation of the resulting graphs and videos, we found that our model got stuck at a local maximum, where the robot learned to sit instead of standing up. We tried to solve this sitting problem by:

Entropy Regularization Tuning: We tried different target entropies values to allow for more exploration.

Reward Normalization: implemented a normalized to scale rewards dynamically to help the agent distinguish the "standing" reward signal from the "sitting" baseline to try to prevent it from settling for the smaller, and easier reward of just sitting

Increased Sample Efficiency: We increased the number of gradient updates per environment step from 4 to 12 (updates_per_step), allowing the model to learn more aggressively from its experiences.

We expanded the neural network architecture to [512, 512, 256] (from [256, 256]) to give the model enough capacity to capture the complex kinematics of the 17-joint humanoid.

Reward Shaping: We experimented with adding extra rewards to encourage the robot to stand up, such as increasing the reward weight of the torso height in the reward and adding rewards based on the difference in z coordinates between different body parts (ex: rewarding the thighs being above the feet).

Due to the significant computational cost of training, in some cases requiring 10+ hours of training, we were not able to explore as many hyperparameters and architectural variations as we had wished. Despite these constraints, the foundational work is solid and given more time, a solution is likely.

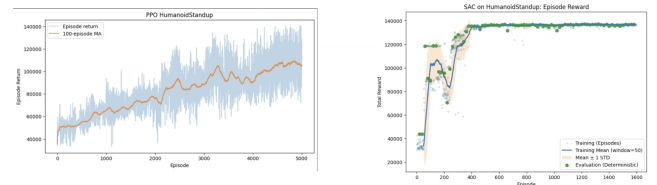


Fig. 6. Episode vs Return For PPO and SAC

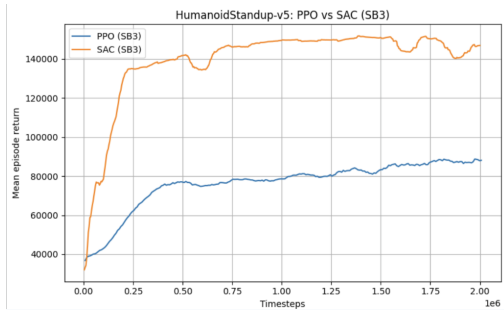


Fig. 7. PPO vs SAC

DISCUSSION & CONCLUSION

Our application of Proximal Policy Optimization and Soft Actor-Critic has revealed multiple important insights. Firstly, ensuring that an agent continues to learn and does not get stuck in a local maximum is a difficult task, especially when trying to teach it a complex task, like standing up, that requires multiple coordinated movements. It is difficult to ensure the agent experiments with these movements when the rewards for it are limited in comparison to the more simple task of sitting up.

Even though we were not successful in completing the task of making the humanoid stand, the project overall gave us hands-on experience on training agents in high degree freedom robotics simulations and a better understanding of deep reinforcement learning fundamentals.

TEAM CONTRIBUTIONS

- Juan: Worked on the fine-tuning of the SAC implementation. Tried different hyper parameters such as Entropy Regularization Tuning, updates per step, epochs, and neural network capacities. Tried different architectures such as adding reward shaping and reward normalization.
- Sravan: Designed and implemented SAC architecture for the standup task, including actor-critic networks and replay buffer training loop. Also configured and tuned continuous action scaling for 17 joints to ensure stable control behavior.
- Tyler: Experimented with multiple architecture configurations and nonlinearities for both PPO and SAC. Experimented with additional reward shaping.
- Kirubiel: Implemented clipped PPO from scratch using PyTorch. Tuned PPO hyperparameters including actor-critic architectures, entropy coefficients, and learning rates. Implemented and ran experiments for PPO and SAC using Stable-Baseline3 to benchmark and validate from scratch algorithms. Experimented with reward shaping to increase returns.

APPENDIX

PPO Stable Baseline 3 Results:

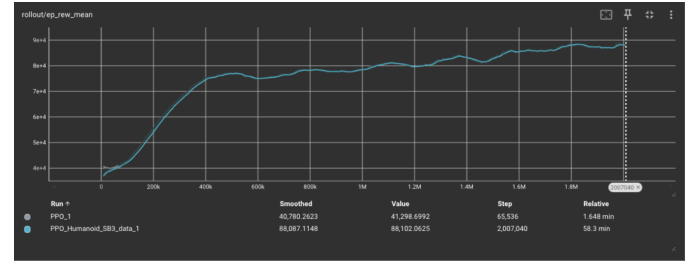


Fig. 8. Episode Reward Mean

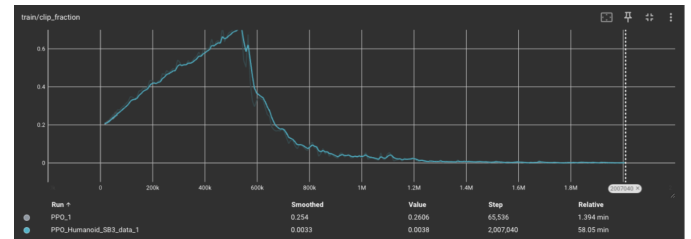
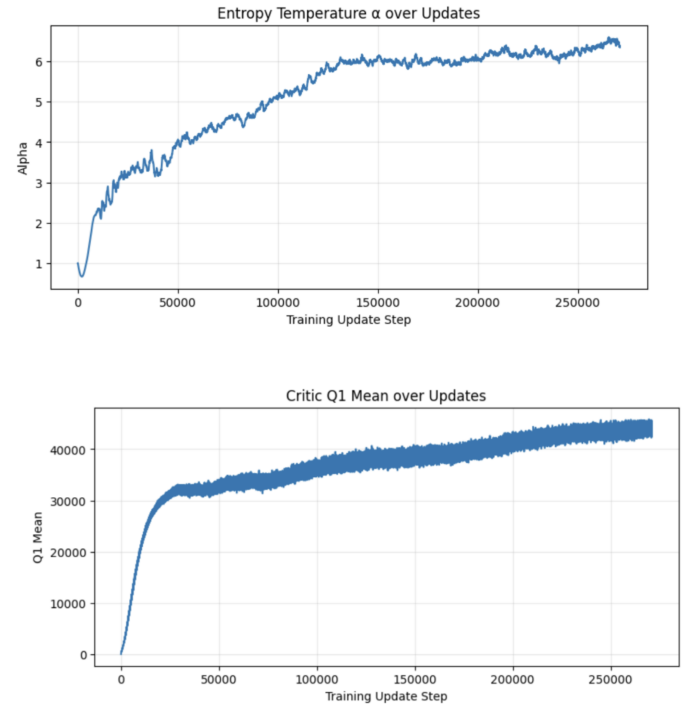


Fig. 9. Train/Clip Fraction

SAC Results



SAC Stable Baseline 3 Results

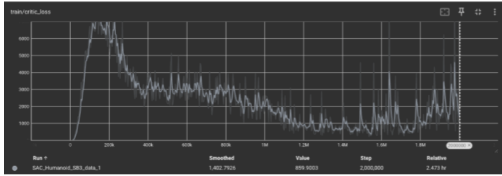


Fig. 10. Train Critic Loss

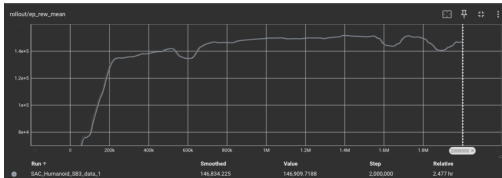


Fig. 11. SAC Episode Reward Mean

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint* arXiv:1707.06347, 2017. [Online]. Available: <https://arxiv.org/pdf/1707.06347>.
- [2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR, vol. 80, pp. 1861–1870, 2018. [Online]. Available: <https://proceedings.mlr.press/v80/haarnoja18b/haarnoja18b.pdf>.
- [3] Stable-Baselines3, “PPO — Proximal Policy Optimization,” *Stable-Baselines3 Documentation*. [Online]. Available: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>. Accessed: Dec. 2, 2025.
- [4] Stable-Baselines3, “SAC — Soft Actor-Critic,” *Stable-Baselines3 Documentation*. [Online]. Available: <https://stable-baselines3.readthedocs.io/en/master/modules/sac.html>. Accessed: Dec. 2, 2025.